DBMS Mini Project

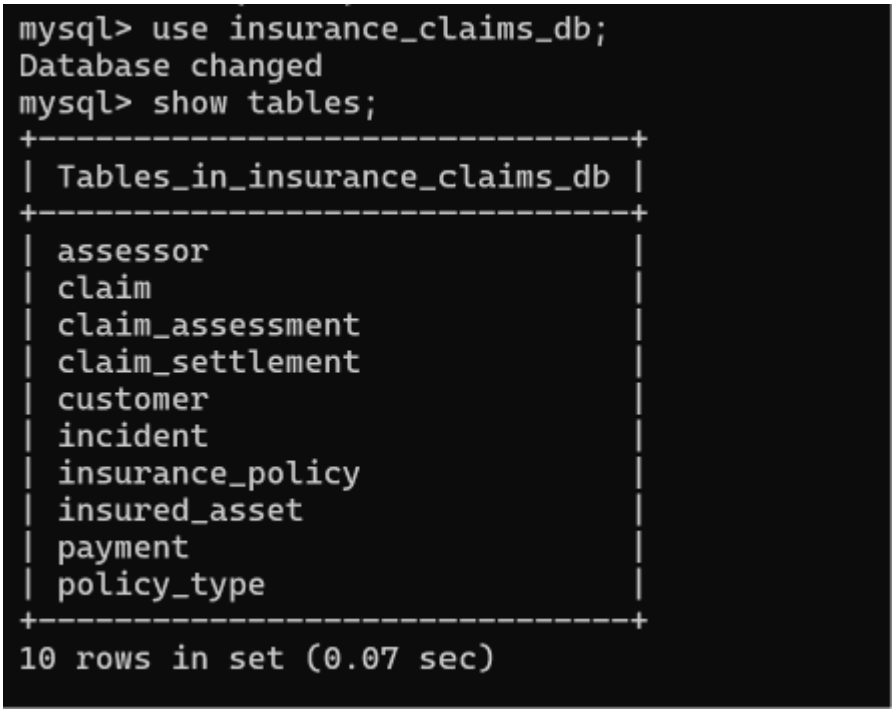
Team 1

Members: Shreya Shenoy PES2UG24CS487

Shreya Patil PES2UG24CS484

Topic: Insurance Claims database

Tables in the db:



1) database created by one teammate and all access being granted to the other

```
mysql> CREATE USER 'shreyap'@'%' IDENTIFIED BY 'shreyap170606';
Query OK, 0 rows affected (0.14 sec)

mysql> GRANT ALL PRIVILEGES
    -> ON insurance_claims_db.*
    -> TO 'shreyap'@'%';
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW GRANTS FOR 'shreyap'@'%';
+-------------------------------------------------------------------+
| Grants for shreyap@%                                              |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `shreyap`@`%`                               |
| GRANT ALL PRIVILEGES ON `insurance_claims_db`.* TO `shreyap`@`%` |
+-------------------------------------------------------------------+
2 rows in set (0.01 sec)

mysql>
```

2) our database and prepopulated tables

```
mysql> USE insurance_claims_db;
Database changed
mysql> SHOW TABLES;
+-----------------------------------+
| Tables_in_insurance_claims_db     |
+-----------------------------------+
| assessor                          |
| claim                             |
| claim_assessment                  |
| claim_settlement                  |
| customer                          |
| incident                          |
| insurance_policy                  |
| insured_asset                     |
| payment                           |
| policy_type                       |
+-----------------------------------+
10 rows in set (0.18 sec)

mysql>
```

Tasks Queries :

Teammate 1: Shreya Shenoy

1)using INNER JOIN and multiple table JOIN concept ,display policies with customer and policy type

```
mysql> SELECT
    ->     p.policy_id,
    ->     c.name AS customer_name,
    ->     pt.policy_name,
    ->     p.start_date,
    ->     p.end_date,
    ->     p.premium_amount,
    ->     p.coverage_amount,
    ->     p.policy_status
    -> FROM INSURANCE_POLICY p
    -> JOIN CUSTOMER c
    ->     ON p.customer_id = c.customer_id
    -> JOIN POLICY_TYPE pt
    ->     ON p.policy_type_id = pt.policy_type_id;
+-----------+------------------+------------------+------------+------------+---------------+-----------------+---------------+
| policy_id | customer_name    | policy_name      | start_date | end_date   | premium_amount | coverage_amount | policy_status |
+-----------+------------------+------------------+------------+------------+---------------+-----------------+---------------+
|      1001 | Shreya Shenoy    | Car Insurance    | 2026-01-01 | 2026-12-31 |       18000.00 |      1000000.00 | Active        |
|      1002 | Shreya Shenoy    | Health Insurance | 2026-01-01 | 2026-12-31 |       22000.00 |      5000000.00 | Active        |
|      1003 | Shreya Patil     | Home Insurance   | 2026-02-01 | 2027-01-31 |       14000.00 |      8000000.00 | Active        |
|      1004 | Shreya Mattimadu | Health Insurance | 2026-01-15 | 2027-01-14 |       25000.00 |      5000000.00 | Active        |
|      1005 | Shreelakshmi Bhat | Car Insurance   | 2026-03-01 | 2027-02-28 |       17000.00 |      1200000.00 | Active        |
|      1006 | Lilas Ikuta      | Travel Insurance | 2026-04-01 | 2027-03-31 |        6000.00 |      1000000.00 | Active        |
+-----------+------------------+------------------+------------+------------+---------------+-----------------+---------------+
6 rows in set (0.17 sec)

mysql>
```

this retrieves insurance policies along with the corresponding customer and policy type details using INNER JOINs.

2)Using concepts of JOIN , GROUP BY , COUNT and HAVING to find customers with multiple policies (using aggregation)

```
mysql> SELECT
    ->     c.customer_id,
    ->     c.name,
    ->     COUNT(p.policy_id) AS number_of_policies
    -> FROM CUSTOMER c
    -> JOIN INSURANCE_POLICY p
    ->     ON c.customer_id = p.customer_id
    -> GROUP BY c.customer_id, c.name
    -> HAVING COUNT(p.policy_id) > 1;
+-------------+---------------+---------------------+
| customer_id | name          | number_of_policies  |
+-------------+---------------+---------------------+
|           1 | Shreya Shenoy |                   2 |
+-------------+---------------+---------------------+
1 row in set (0.02 sec)

mysql>
```

3) Using concepts of aggregate AVG , JOIN and GROUP BY to find average claim amount by policy type

```
mysql> SELECT
    ->     pt.policy_name,
    ->     AVG(cl.claimed_amount) AS average_claim_amount
    -> FROM POLICY_TYPE pt
    -> JOIN INSURANCE_POLICY p
    ->     ON pt.policy_type_id = p.policy_type_id
    -> JOIN CLAIM cl
    ->     ON p.policy_id = cl.policy_id
    -> GROUP BY pt.policy_type_id, pt.policy_name;
+------------------+----------------------+
| policy_name      | average_claim_amount |
+------------------+----------------------+
| Car Insurance    |         550000.000000 |
| Health Insurance |         250000.000000 |
| Home Insurance   |         500000.000000 |
| Travel Insurance |          80000.000000 |
+------------------+----------------------+
4 rows in set (0.04 sec)

mysql>
```

4) Using JOIN to compare the amount claimed with the amount assessed for each claim and calculate the difference

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     ca.assessed_amount,
    ->     (cl.claimed_amount - ca.assessed_amount) AS difference
    -> FROM CLAIM cl
    -> JOIN CLAIM_ASSESSMENT ca
    ->     ON cl.claim_id = ca.claim_id;
+----------+----------------+-----------------+------------+
| claim_id | claimed_amount | assessed_amount | difference |
+----------+----------------+-----------------+------------+
|      501 |      200000.00 |       160000.00 |   40000.00 |
|      502 |      350000.00 |       300000.00 |   50000.00 |
|      503 |      500000.00 |       420000.00 |   80000.00 |
|      504 |      900000.00 |       850000.00 |   50000.00 |
+----------+----------------+-----------------+------------+
4 rows in set (0.04 sec)

mysql>
```

5) Making subqueries and using AVG aggregate function to find claims above the average claim amount

```
mysql> SELECT
    ->     claim_id,
    ->     policy_id,
    ->     claimed_amount,
    ->     claim_status
    -> FROM CLAIM
    -> WHERE claimed_amount > (
    ->     SELECT AVG(claimed_amount)
    ->     FROM CLAIM
    -> );
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      503 |      1003 |      500000.00 | Settled          |
|      504 |      1005 |      900000.00 | Under Assessment |
+----------+-----------+----------------+------------------+
2 rows in set (0.01 sec)

mysql>
```

6)Using JOIN , and arithmetic functions like ROUND to calculate the percentage of the claimed amount that was ultimately settled for each settled claim

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     cs.settlement_amount,
    ->     ROUND(
    ->         (cs.settlement_amount / cl.claimed_amount) * 100,
    ->         2
    ->     ) AS settlement_percentage
    -> FROM CLAIM cl
    -> JOIN CLAIM_SETTLEMENT cs
    ->     ON cl.claim_id = cs.claim_id;
+----------+----------------+-------------------+-----------------------+
| claim_id | claimed_amount | settlement_amount | settlement_percentage |
+----------+----------------+-------------------+-----------------------+
|      501 |      200000.00 |         150000.00 |                 75.00 |
|      502 |      350000.00 |         290000.00 |                 82.86 |
|      503 |      500000.00 |         400000.00 |                 80.00 |
+----------+----------------+-------------------+-----------------------+
3 rows in set (0.05 sec)

mysql>
```

7)using Procedure to get complete claim details about a user by just their ID

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetClaimDetails(IN p_claim_id INT)
   -> BEGIN
   ->     SELECT
   ->         c.name AS customer_name,
   ->         p.policy_id,
   ->         pt.policy_name,
   ->         ia.asset_type,
   ->         i.incident_type,
   ->         cl.claim_id,
   ->         cl.claim_date,
   ->         cl.claimed_amount,
   ->         cl.claim_status,
   ->         ca.assessed_amount,
   ->         cs.settlement_amount,
   ->         pay.payment_amount
   ->     FROM CLAIM cl
   ->     JOIN INSURANCE_POLICY p
   ->         ON cl.policy_id = p.policy_id
   ->     JOIN CUSTOMER c
   ->         ON p.customer_id = c.customer_id
   ->     JOIN POLICY_TYPE pt
   ->         ON p.policy_type_id = pt.policy_type_id
   ->     JOIN INSURED_ASSET ia
   ->         ON p.policy_id = ia.policy_id
   ->     JOIN INCIDENT i
   ->         ON cl.incident_id = i.incident_id
   ->     LEFT JOIN CLAIM_ASSESSMENT ca
   ->         ON cl.claim_id = ca.claim_id
   ->     LEFT JOIN CLAIM_SETTLEMENT cs
   ->         ON cl.claim_id = cs.claim_id
   ->     LEFT JOIN PAYMENT pay
   ->         ON cs.settlement_id = pay.settlement_id
   ->     WHERE cl.claim_id = p_claim_id;
   -> END //
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL GetClaimDetails(501);
+--------------+-----------+--------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| customer_name | policy_id | policy_name   | asset_type | incident_type | claim_id | claim_date | claimed_amount | claim_status | assessed_amount | settlement_amount | payment_amount |
+--------------+-----------+--------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| Shreya Shenoy |      1001 | Car Insurance | Car        | Car Accident  |      501 | 2026-08-21 |      200000.00 | Settled      |       160000.00 |         150000.00 |      150000.00 |
+--------------+-----------+--------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
1 row in set (0.20 sec)

Query OK, 0 rows affected (0.25 sec)
```

```
mysql> CALL GetClaimDetails(503);
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| customer_name | policy_id | policy_name   | asset_type | incident_type | claim_id | claim_date | claimed_amount | claim_status | assessed_amount | settlement_amount | payment_amount |
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| Shreya Patil  |      1003 | Home Insurance | House      | Property Damage |      503 | 2026-06-11 |      500000.00 | Settled      |       420000.00 |         400000.00 |      400000.00 |
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql>
```

Teammate 2: Shreya Patil

Able to connect to the database using ip address and port number. Both systems need to be on the same network to do so.

```
C:\Users\shrey>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.30.95.233 -P 3306 -u shreyap -p
Enter password: *************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show tables;
ERROR 1046 (3D000): No database selected
mysql> use insurance_claims_db;
Database changed
mysql> show tables;
+--------------------------------+
| Tables_in_insurance_claims_db |
+--------------------------------+
| assessor                       |
| claim                          |
| claim_assessment               |
| claim_settlement               |
| customer                       |
| incident                       |
| insurance_policy               |
| insured_asset                  |
| payment                        |
| policy_type                    |
+--------------------------------+
10 rows in set (0.07 sec)
```

1) Finding claims above ₹1,00,000

```
mysql> SELECT
    ->     claim_id,
    ->     policy_id,
    ->     claimed_amount,
    ->     claim_status
    -> FROM CLAIM
    -> WHERE claimed_amount > 100000
    -> ORDER BY claimed_amount DESC;
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      504 |      1005 |      900000.00 | Under Assessment |
|      503 |      1003 |      500000.00 | Settled          |
|      502 |      1004 |      350000.00 | Settled          |
|      501 |      1001 |      200000.00 | Settled          |
|      505 |      1002 |      150000.00 | Approved         |
+----------+-----------+----------------+------------------+
5 rows in set (0.24 sec)
```

2)Counting all the claims by status

```
mysql> SELECT
    ->     claim_status,
    ->     COUNT(*) AS number_of_claims
    -> FROM CLAIM
    -> GROUP BY claim_status;
+------------------+------------------+
| claim_status     | number_of_claims |
+------------------+------------------+
| Settled          |                3 |
| Under Assessment |                1 |
| Approved         |                1 |
| Under Review     |                1 |
+------------------+------------------+
4 rows in set (0.02 sec)
```

3)Calculates the total claimed amount for each policy type and orders the results from highest to lowest.

```
mysql> SELECT
    ->     pt.policy_name,
    ->     SUM(cl.claimed_amount) AS total_claimed_amount
    -> FROM POLICY_TYPE pt
    -> JOIN INSURANCE_POLICY p
    ->     ON pt.policy_type_id = p.policy_type_id
    -> JOIN CLAIM cl
    ->     ON p.policy_id = cl.policy_id
    -> GROUP BY pt.policy_type_id, pt.policy_name
    -> ORDER BY total_claimed_amount DESC;
+-----------------+----------------------+
| policy_name     | total_claimed_amount |
+-----------------+----------------------+
| Car Insurance   |           1100000.00 |
| Health Insurance |            500000.00 |
| Home Insurance  |            500000.00 |
| Travel Insurance |             80000.00 |
+-----------------+----------------------+
4 rows in set (0.03 sec)
```

4)Identifies claims where the amount assessed by the assessor is lower than the amount originally claimed.

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     ca.assessed_amount
    -> FROM CLAIM cl
    -> JOIN CLAIM_ASSESSMENT ca
    ->     ON cl.claim_id = ca.claim_id
    -> WHERE ca.assessed_amount < cl.claimed_amount;
+----------+----------------+-----------------+
| claim_id | claimed_amount | assessed_amount |
+----------+----------------+-----------------+
|      501 |      200000.00 |       160000.00 |
|      502 |      350000.00 |       300000.00 |
|      503 |      500000.00 |       420000.00 |
|      504 |      900000.00 |       850000.00 |
+----------+----------------+-----------------+
4 rows in set (0.16 sec)
```

5)Identifies claims that do not yet have an associated settlement using a LEFT JOIN and NULL filtering

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.policy_id,
    ->     cl.claimed_amount,
    ->     cl.claim_status
    -> FROM CLAIM cl
    -> LEFT JOIN CLAIM_SETTLEMENT cs
    ->     ON cl.claim_id = cs.claim_id
    -> WHERE cs.settlement_id IS NULL;
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      504 |      1005 |      900000.00 | Under Assessment |
|      505 |      1002 |      150000.00 | Approved         |
|      506 |      1006 |       80000.00 | Under Review     |
+----------+-----------+----------------+------------------+
3 rows in set (0.15 sec)
```

6)Identifies settlements for which the corresponding payment has not yet been made

```
mysql> SELECT
    ->     cs.settlement_id,
    ->     cs.claim_id,
    ->     cs.settlement_amount,
    ->     cs.settlement_status
    -> FROM CLAIM_SETTLEMENT cs
    -> LEFT JOIN PAYMENT p
    ->     ON cs.settlement_id = p.settlement_id
    -> WHERE p.payment_id IS NULL;
Empty set (0.17 sec)
```

7) Calculate settlement percentage using the concept of settlement amount ÷ claimed amount × 100 by creating a function

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION CalculateSettlementPercentage(p_claim_id INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_claimed_amount DECIMAL(15,2);
    ->     DECLARE v_settlement_amount DECIMAL(15,2);
    ->
    ->     SELECT
    ->         cl.claimed_amount,
    ->         cs.settlement_amount
    ->     INTO
    ->         v_claimed_amount,
    ->         v_settlement_amount
    ->     FROM CLAIM cl
    ->     LEFT JOIN CLAIM_SETTLEMENT cs
    ->         ON cl.claim_id = cs.claim_id
    ->     WHERE cl.claim_id = p_claim_id;
    ->
    ->     IF v_claimed_amount IS NULL
    ->        OR v_settlement_amount IS NULL THEN
    ->         RETURN NULL;
    ->     END IF;
    ->
    ->     RETURN ROUND(
    ->         (v_settlement_amount / v_claimed_amount) * 100,
    ->         2
    ->     );
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT CalculateSettlementPercentage(501);
+-------------------------------------+
| CalculateSettlementPercentage(501) |
+-------------------------------------+
|                              75.00 |
+-------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CalculateSettlementPercentage(502);
+-------------------------------------+
| CalculateSettlementPercentage(502) |
+-------------------------------------+
|                              82.86 |
+-------------------------------------+
1 row in set (0.01 sec)

mysql> |
```

8) using a trigger function to automatically update claim status after settlement

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_claim_settlement
    -> AFTER INSERT ON CLAIM_SETTLEMENT
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE CLAIM
    ->     SET claim_status = 'Settled'
    ->     WHERE claim_id = NEW.claim_id;
    -> END //
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

checking claim's current status

```
mysql> DELIMITER ;
mysql> SELECT claim_id, claim_status
    -> FROM CLAIM
    -> WHERE claim_id = 501;
+----------+----------------+
| claim_id | claim_status   |
+----------+----------------+
|      501 | Settled        |
+----------+----------------+
1 row in set (0.01 sec)

mysql>
```